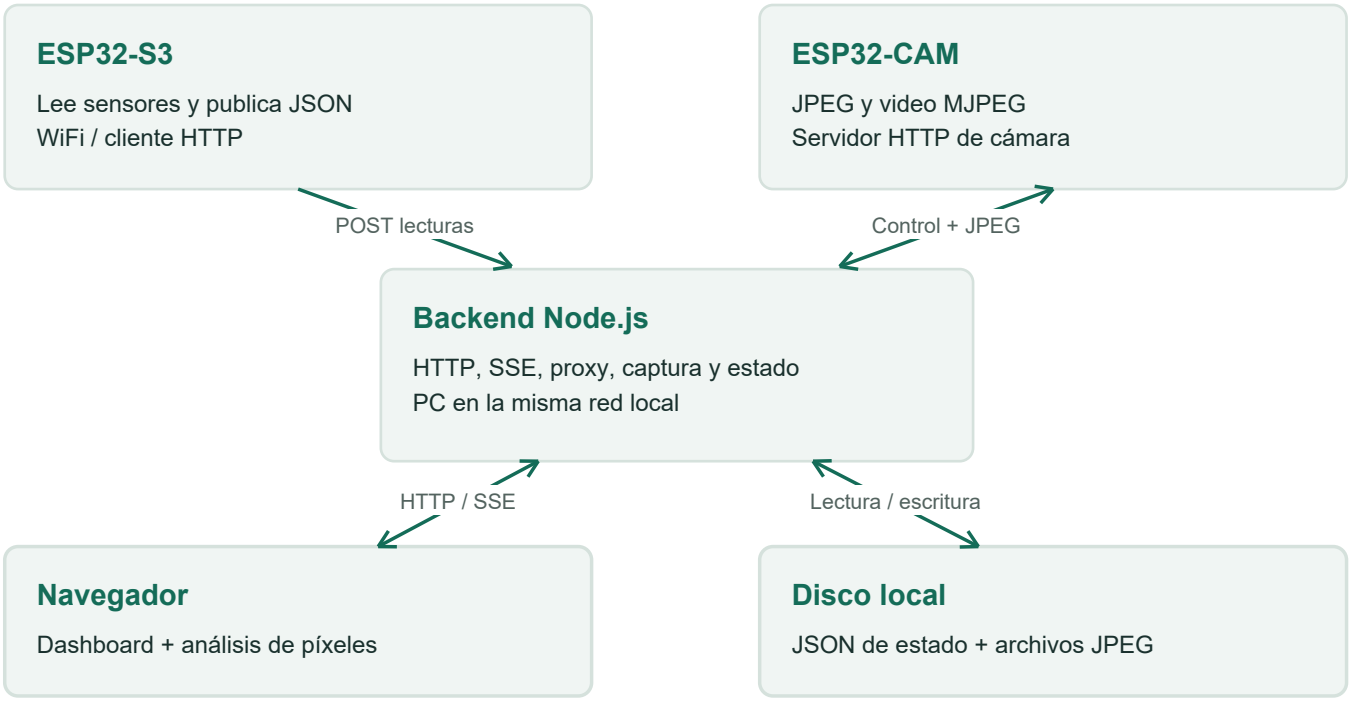
PROYECTO DE MONITOREO HIDROPÓNICO

HydroVision

Documentación técnica integral

Cómo está construida la aplicación web, cómo se comunica con los microcontroladores y cómo transforma fotografías de hojas en indicadores visuales.



Edición: 15 de septiembre de 2026
Base: código del espacio de trabajo WebHidroponia; commit de referencia 0397600 más cambios locales.
Alcance: implementación, fundamentos, interfaces, operación, pruebas y límites. No contiene contraseñas ni claves de acceso.

MEMORIA TÉCNICA / 01

Guía de lectura y alcance

Este informe reconstruye la arquitectura a partir de los archivos del proyecto. Describe el sistema que existe hoy, no una arquitectura idealizada ni una narración de decisiones históricas que no estén documentadas. Cuando se explica una ventaja de una técnica, se trata de una interpretación técnica de su uso en el código.

Tres niveles que conviene distinguir

Nivel	Qué significa
Implementado	Hay código ejecutable que realiza la función. Esto no demuestra que cada dispositivo físico esté conectado o calibrado.
Validado en pruebas	El comportamiento se ejercitó en casos automatizados o está registrado en la documentación de ensayos anteriores.
Propuesto	Es una mejora recomendada. No debe describirse como parte del sistema ya construido.

La versión del paquete es **1.0.0**; la versión específica del algoritmo es **leaf-pixel-v4.4**. Son identificadores distintos: cambiar el detector no actualiza automáticamente la versión del paquete. La referencia Git revisada es **0397600**, del 7 de septiembre de 2026. Hay cambios locales posteriores en visión, interfaz, pruebas y documentación que forman parte de este informe.

Qué no se hizo para redactarlo

No se reprogramaron las placas, no se modificó la aplicación ni se cambiaron umbrales. No se abrió el acceso a Internet, no se subieron archivos a Git y no se encendió el flash. Las pruebas ejecutadas usan imágenes ya guardadas y servidores HTTP de ensayo, sin exigir una captura nueva de la planta.

> **Límite central**
> El sistema encuentra patrones visuales compatibles con manchas y amarillamiento. No identifica especies de plagas ni diagnostica enfermedades. La estimación de área proyectada no es una medición física de crecimiento.

Recorrido recomendado

Para entender el conjunto: arquitectura, tecnologías y frontend. Para integrar dispositivos: firmware, protocolos y API. Para defender el método de análisis: segmentación, manchas, amarillo, nitidez y comparación temporal. Para mantener o publicar el proyecto: persistencia, pruebas, seguridad, operación y evolución.

MEMORIA TÉCNICA / 02

Índice de contenidos

Los marcadores del PDF permiten saltar a cada capítulo. La numeración de esta tabla corresponde a la página del documento.

2 Guía de lectura y alcance

4 Arquitectura general

5 Lenguajes, tecnologías y dependencias

6 Estructura del repositorio

7 Frontend: estructura y diseño visual

8 Frontend: estado, eventos y actualización

9 Frontend: cámara, ROI y superposición

10 Backend: ciclo de vida y responsabilidades

11 Modelo de datos y reglas de alarmas

12 Catálogo de sensores y cobertura física

13 Firmware de sensores: funcionamiento

14 Conversión de mediciones y calibración

15 Firmware de cámara y configuración

16 Protocolos y comunicaciones

17 API de lectura, estado e ingesta

18 API de cámara, visión y controles

19 Captura reproducible: secuencia completa

20 Parser MJPEG, concurrencia y recuperación

21 Visión: preparación y segmentación foliar

22 Visión: detección de manchas

23 Visión: amarillamiento de hojas

24 Nitidez, calidad y foco: qué mide el software

25 Fusión de tres fotos y repetibilidad

26 Comparación de área y crecimiento

27 Persistencia, imágenes y trazabilidad

28 Patrones y decisiones de diseño

29 Configuración del servidor

30 Instalación y puesta en marcha

31 Pruebas y evidencia de validación

32 Seguridad y límites antes de publicar

33 Operación, fallos y diagnóstico

34 Rendimiento, escalabilidad y despliegue

35 Evolución recomendada del proyecto

36 Glosario técnico

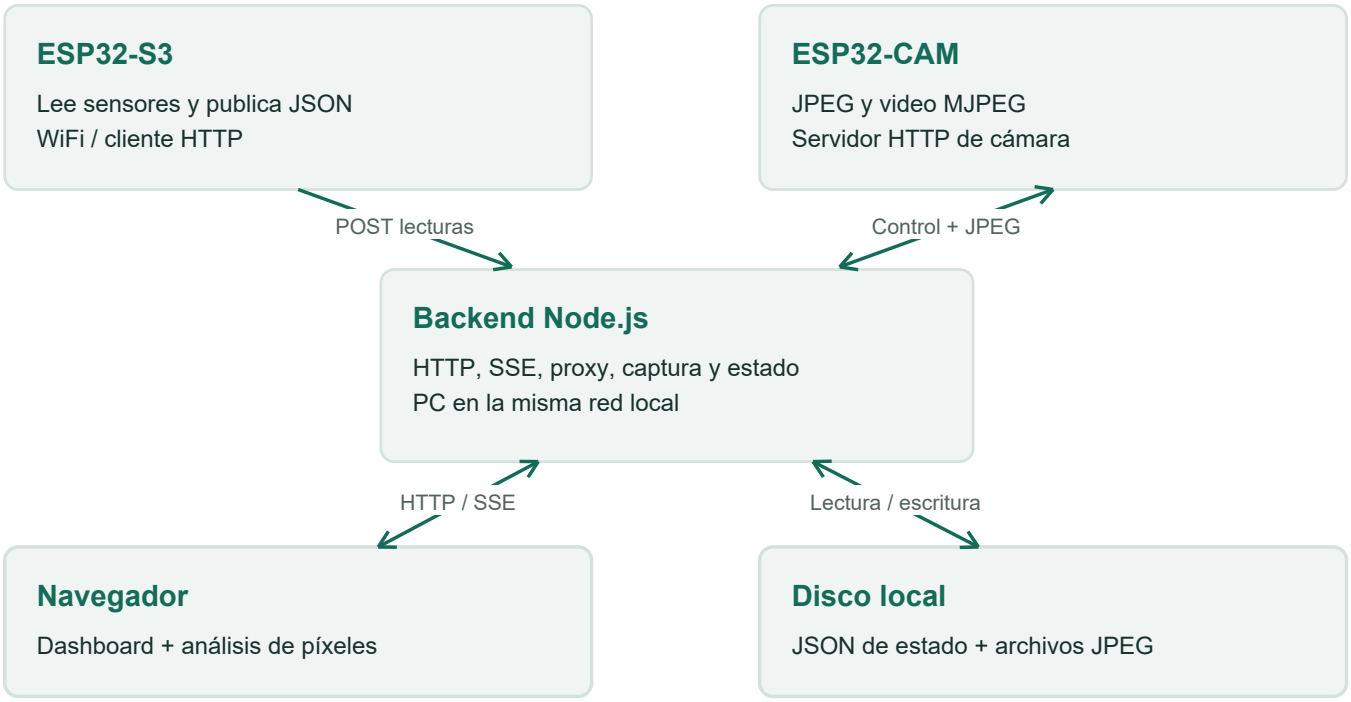
37 Referencias y mapa de lectura del código

MEMORIA TÉCNICA / 03

Arquitectura general

Base de implementación: server.js; app/scripts/app.js; firmware/

HydroVision es una aplicación cliente-servidor con dos dispositivos embebidos. La ESP32-S3 mide y envía datos; la ESP32-CAM ofrece imágenes; Node.js concentra las comunicaciones y conserva el estado; el navegador presenta el dashboard y ejecuta el análisis de hojas.



Dos flujos independientes

Telemetría: la ESP32-S3 inicia una petición HTTP hacia Node. El servidor normaliza las lecturas, actualiza el histórico y emite un evento SSE. La página cambia los valores sin recargarse. La cámara no participa en este recorrido.

Visión: el navegador solicita una captura; Node configura temporalmente la cámara y guarda tres JPEG. El navegador descarga esas fotos y calcula máscaras, calidad y repetición. Después envía las métricas al servidor para persistirlas. La ESP32-CAM no ejecuta el detector de manchas.

Límites de cada componente

La PC debe permanecer encendida y accesible para servir el dashboard y recibir sensores. El USB se usa para alimentación, programación y diagnóstico serial; el intercambio operativo descrito por el firmware se realiza por WiFi. No hay base de datos externa, broker de mensajes ni servicio de inferencia en la nube.

> **Disponibilidad no equivale a conexión**
> Una URL de cámara configurada hace aparecer su servicio como online en el estado del servidor. Esa etiqueta no es una prueba periódica de conectividad con el dispositivo.

MEMORIA TÉCNICA / 04

Lenguajes, tecnologías y dependencias

Base de implementación: package.json; index.html; sketch.yaml de ambas placas

Tecnología	Responsabilidad real
HTML5	Define la estructura del dashboard, secciones, controles, imágenes, iframe y canvas.
CSS3	Diseña colores, tipografía, grillas, estados y adaptación a pantallas pequeñas.
JavaScript del navegador	Maneja eventos, fetch, SSE, dibujo y procesamiento de píxeles.
JavaScript en Node.js	Implementa HTTP, archivos estáticos, API, persistencia, proxy y captura.
C++ / Arduino	Programa los dos microcontroladores, sensores, WiFi y servidor de cámara.
JSON	Transporta lecturas y métricas; también representa el estado persistido.
YAML	Describe perfiles de compilación Arduino en sketch.yaml.
JPEG / MJPEG	Fotografías comprimidas y sucesión de imágenes sobre HTTP.

No se utilizan React, Vue, Angular, Express, TypeScript, Tailwind ni un empaquetador como Vite. El puerto 4173 no implica que Vite esté instalado. Node sirve directamente index.html y los archivos de app/. Tampoco hay Chart.js: los gráficos se dibujan con Canvas 2D propio.

Dependencias separadas por entorno

El backend usa módulos incluidos en Node: http, https, fs, path y URL. El paquete no declara dependencias externas. Las pruebas de imágenes reales y utilidades de auditoría pueden usar **sharp**, que es opcional y no forma parte del funcionamiento básico de la web.

Los perfiles Arduino fijan el core **esp32:esp32 3.3.8**. El perfil de sensores declara **DHT sensor library 1.4.7** y **Adafruit Unified Sensor 1.1.15**. La cámara utiliza esp_camera y el servidor HTTP del entorno Espressif incluido en ese ecosistema.

Reproducibilidad pendiente
package.json no fija una versión de Node mediante engines. Conviene registrar la versión utilizada y documentar la instalación de sharp para que todas las pruebas puedan repetirse en otra PC.

MEMORIA TÉCNICA / 05

Estructura del repositorio

Base de implementación: Árbol de archivos del espacio de trabajo

```
WebHidroponia/
  index.html
  server.js
  package.json
  app/
    camera-burst.js
    scripts/
      app.js
      charts.js
      plant-vision.js
      plant-trends.js
    styles/main.css
    docs/
  firmware/
    esp32_web_direct/
    esp32_cam_webserver/
  tests/
  data/                 # datos locales, parcialmente ignorados
  output/pdf/           # documento generado en esta tarea
```

Archivo o grupo	Responsabilidad
server.js	Punto de entrada del backend; estado compartido y despacho de rutas.
app/scripts/app.js	Coordinador de interfaz, datos, cámara, selección y análisis.
plant-vision.js	Funciones del motor de segmentación y fusión de fotografías.
plant-trends.js	Comparación histórica bajo condiciones compatibles.
charts.js	Gráficos de líneas y animación de cámara simulada.
camera-burst.js	Parser multipart, adquisición acotada y restauración de parámetros.
firmware/	Código compilado y cargado por separado en cada placa.
tests/	Pruebas con node:test y herramientas auxiliares de diagnóstico.

Es una separación por responsabilidades y entorno de ejecución, no un monorepo de microservicios. Los archivos del motor visual se comparten entre navegador y pruebas Node sin duplicar su implementación.

MEMORIA TÉCNICA / 06

Frontend: estructura y diseño visual

Base de implementación: index.html; app/styles/main.css; app/scripts/charts.js

HTML semántico y superficies del dashboard

La página organiza una vista general de sensores, cámara, análisis foliar, alarmas, operación, histórico e integración. Las secciones tienen encabezados y referencias aria-labelledby. Los controles son botones HTML y la visibilidad se alterna con hidden y clases CSS. Hay etiquetas accesibles en canvas y una región aria-live para sensores, aunque no existe una auditoría integral de accesibilidad documentada.

CSS sin framework

Las variables en :root definen fondo, paneles, texto, acentos, estados y radio de borde. Esto actúa como un pequeño sistema de tokens visuales. La familia tipográfica parte de Avenir Next y continúa con Segoe UI y sans-serif; valores técnicos usan fuentes monoespaciadas como Consolas.

La estructura combina CSS Grid, Flexbox y tamaños mínimos/máximos. El contenedor principal se limita a 1440 px y conserva márgenes. Las tarjetas de sensores usan repeat(auto-fit, minmax(220px, 1fr)); los gráficos usan un mínimo de 280 px. Las reglas a 1080 y 720 px reorganizan columnas para pantallas más pequeñas.

Los estados visuales se expresan con variantes de clases: una lectura normal no tiene el mismo tratamiento que una advertencia o alarma crítica. La hoja de estilos real tiene paneles oscuros, verdes y acentos cálidos; no depende de una biblioteca de componentes instalada.

Gráficos propios sobre Canvas 2D

drawLineChart limpia el lienzo, dibuja una retícula horizontal, normaliza valores y traza segmentos entre muestras. El último dato se destaca con un punto. El eje horizontal reparte muestras por índice, no por separación temporal real: si las llegadas son irregulares, la distancia visual no representa exactamente el tiempo transcurrido.

```
x = padding + (width - 2 * padding) * index / (points.length - 1)
y = height - padding - (value - min) / (max - min)
    * (height - 2 * padding)
```

drawCamera genera únicamente una animación de demostración. No es video de la instalación ni participa del análisis real. La cámara real usa un elemento img para JPEG/MJPEG o un iframe cuando se configura un reproductor externo.

MEMORIA TÉCNICA / 07

Frontend: estado, eventos y actualización

Base de implementación: app/scripts/app.js: bootstrap, hydrate, connectStream

app.js está encapsulado en una función autoejecutable o IIFE. Sus variables internas conservan el estado de la interfaz sin publicarlo todo en window. Obtiene referencias al DOM una vez y mantiene mapas de tarjetas, gráficos y actuadores para actualizar elementos existentes.

Secuencia de inicio

Paso	Comportamiento
1. bootstrap	Solicita GET /api/bootstrap con fetch y verifica la respuesta.
2. hydrate	Toma sensores, snapshot, history, alarmas, servicios y cámara; construye o actualiza el DOM.
3. connectStream	Abre EventSource en /api/stream y escucha eventos llamados snapshot.
4. Nueva lectura	Cada evento contiene el estado público completo; hydrate vuelve a renderizar sus partes.
5. Fallo inicial	Crea un mock local, actualiza cada 2 segundos y reintenta bootstrap cada 10 segundos.

Si falla el canal SSE después de conectar, la página muestra un estado de reconexión y utiliza el comportamiento de EventSource. No se implementa una cola local de eventos ni reproducción por identificador Last-Event-ID. Al volver a conectar se recibe el estado actual, no todos los eventos perdidos.

Separación de estados de cámara y análisis

currentCameraMode representa mock, mjpeg o embed. currentCameraViewMode distingue imagen de video. plantVisionBusy impide iniciar dos análisis desde la misma vista. La configuración del servidor incluye analysisInProgress para que otras pestañas también conozcan que la cámara está ocupada.

Los botones llaman endpoints específicos y el estado recibido por SSE actualiza la interfaz. No hay Redux, store reactivo ni enlace bidireccional automático. La coordinación se escribe explícitamente mediante funciones y escuchadores addEventListener.

> **El modo local es una demostración**
> Si no está disponible el backend, los números que aparecen pueden provenir del generador del navegador. Debe leerse la etiqueta de origen; ver una tarjeta actualizándose no prueba que haya una medición física reciente.

Persistencia del cliente

La selección de hoja se guarda dentro de las métricas del análisis en el servidor. No se utiliza localStorage como base de datos de la aplicación. Al recuperar una lectura, la ROI se restaura antes de descargar sus fotos para evitar una carrera con un análisis nuevo.

MEMORIA TÉCNICA / 08

Frontend: cámara, ROI y superposición

Base de implementación: app/scripts/app.js: renderCamera, drawPlantPreview, runPlantVisionAnalysis

El modo imagen solicita una foto de alta resolución mediante el proxy del backend. El modo video conecta el elemento img a una secuencia MJPEG. La actualización de foto utiliza un parámetro de URL para evitar reutilizar una respuesta cacheada. El valor de refresco predeterminado del servidor es una hora; el usuario puede pedir una foto manualmente.

Qué ocurre al presionar Analizar

Se deshabilitan controles que podrían interferir, se detiene el refresco y se quita temporalmente la fuente del video. El servidor devuelve tres referencias de captura. loadPlantImage las decodifica y analyzePlantFrames dibuja cada una en un canvas de muestreo para obtener RGBA. combine elige la referencia y consolida evidencia. makePlantAnalysis arma el objeto que se envía para guardar.

Selección de región de interés

La ROI se define con pointerdown, pointermove y pointerup. Sus coordenadas se normalizan al intervalo 0 a 1 respecto de la imagen completa. Un cambio de tamaño de pantalla no cambia el significado de esas coordenadas. Las selecciones diminutas, menores que el mínimo previsto por la interfaz, se descartan; el motor también valida límites y dimensiones.

```
roi = { x: 0.20, y: 0.10, width: 0.55, height: 0.75 }
// Ejemplo ilustrativo, no selección actual del usuario.
pixelX = roi.x * sampleWidth
pixelY = roi.y * sampleHeight
```

Restablecer el área puede reutilizar las fotos en memoria y volver a analizarlas; no necesariamente obtiene una nueva ráfaga. focusCrop amplía la región conservando su proporción. Al seleccionar, la vista vuelve a mostrar la imagen completa para no mezclar coordenadas del recorte visual con las de análisis.

Marca	Significado
Rojo #ee4858	Manchas que superan las reglas de forma y repetición.
Naranja oscuro #c45a10	Superficie amarillenta repetida y cuantificable.
Celeste #53bde0	Candidatas cuya evidencia todavía es insuficiente.

La capa de color se dibuja sobre la fotografía sin modificar sus píxeles originales. El rojo tiene prioridad si coincide con amarillo. Se puede ocultar la superposición y abrir la foto original. Una ampliación visual no recupera detalles ópticos que no fueron capturados.

MEMORIA TÉCNICA / 09

Backend: ciclo de vida y responsabilidades

Base de implementación: server.js

Inicialización de un proceso único

Al ejecutar node server.js, Node carga los módulos estándar, lee variables de entorno, define el catálogo de sensores y recupera el estado local. Después reconstruye actuadores y servicios, persiste el estado, registra temporizadores y abre el servidor HTTP. PORT vale 4173 si no se configura otro valor.

http.createServer recibe cada request y decide qué función invocar según método y pathname. Las rutas de cámara se comprueban antes que archivos estáticos. sendJson centraliza Content-Type, Content-Length y Cache-Control: no-store. La API no está implementada como controladores de Express ni como funciones serverless.

Estado central y trabajo asíncrono

El objeto state vive en memoria dentro del proceso. Las operaciones de red usan promesas y eventos; las escrituras de estado se hacen con fs.writeFileSync. Esto simplifica la consistencia en un único proceso, pero una escritura grande bloquea temporalmente su event loop. No hay coordinación entre múltiples instancias del backend.

Temporización	Uso
2 segundos	tick del simulador del servidor, solo si el modo mock está activo y ejecutándose.
15 segundos	Comentario keep-alive en conexiones SSE.
15 segundos	Umbral LIVE_STALE_MS al calcular antigüedad de la ingesta.
40 muestras	Límite de histórico por sensor.
20 alarmas	Máximo de eventos de alarma conservados.
40 análisis	Máximo de entradas de visión en el estado.

Manejo de errores

Las capturas responden con códigos explícitos como 409 cuando hay un análisis pendiente y 502 cuando falla la cámara. El enrutador tiene un catch general que responde 500. parseBody convierte JSON inválido en un objeto vacío; no existe un esquema estricto compartido ni un límite de cuerpo en esa función.

Detalle de actualización pendiente
La antigüedad se calcula al reconstruir servicios. En modo real tick retorna sin reconstruirlos; si dejan de llegar datos, una etiqueta previamente online puede quedar antigua hasta otra actualización. El umbral existe, pero no equivale a un monitor periódico completo.

MEMORIA TÉCNICA / 10

Modelo de datos y reglas de alarmas

Base de implementación: server.js: sensors, formatReading, toStatus, getPublicState

```
{
  "sensorId": "air-temperature",
  "label": "Temperatura ambiente",
  "value": 24.5,
  "unit": "°C",
  "status": "ok",
  "timestamp": "2026-09-15T19:00:00.000Z",
  "source": "esp32-direct",
  "channel": "huerta/sensores/ambiente/temperatura"
}
```

Ejemplo de contrato, no lectura obtenida durante la redacción. El servidor agrega etiqueta, unidad, estado y canal a partir del catálogo.

Campo de estado	Contenido
sensors	Metadatos: identificador, unidad, decimales, rangos y color.
snapshot / history	Última lectura por sensor y arreglos de muestras recientes.
alarms	Eventos recientes con severidad, título, detalle y hora.
actuators / services	Estados de operación y componentes de la aplicación.
meta	updatedAt, running, dataMode y lastIngestAt.
plantVision	latest e history con fotos referenciadas y métricas.
manualOverrides / remoteActuators	Estado interno de simulación o datos de operación recibidos.

Regla de clasificación

Un dato dentro de idealMin e idealMax es ok. Fuera de ese intervalo es warning, salvo que supere un límite crítico calculado como el 45% del tramo entre el rango ideal y el extremo del catálogo. No es un sistema de diagnóstico agronómico ni una regla universal por especie.

```
criticalLow  = idealMin - (idealMin - min) * 0.45
criticalHigh = idealMax + (max - idealMax) * 0.45
```

Por ejemplo, para pH los valores de catálogo producen límites críticos de 5.215 y 7.175. Las comparaciones críticas son estrictas: menor o mayor, no igualdad. Cada muestra fuera de ideal puede generar un evento; no hay histéresis, confirmación temporal ni reconocimiento manual de alarmas. min y max limitan el simulador, pero no se usan para rechazar automáticamente una lectura real fuera de escala.

MEMORIA TÉCNICA / 11

Catálogo de sensores y cobertura física

Base de implementación: server.js: sensors; firmware/esp32_web_direct/esp32_web_direct.ino

Indicador	Unidad / ideal del código	Adquisición en este firmware
Humedad ambiente	% / 55 a 75	DHT11, habilitado.
Temperatura ambiente	°C / 20 a 28	DHT11, habilitado.
Nivel de agua	% / 55 a 95	HC-SR04; respaldo analógico.
CO2 ambiente	ppm / 500 a 1200	No implementada.
Temperatura del agua	°C / 18 a 24	No implementada.
pH del agua	pH / 5.8 a 6.5	Código opcional, deshabilitado.
Intensidad lumínica	lux / 18000 a 28000	No implementada.
Conductividad eléctrica	mS/cm / 1.2 a 2.4	No implementada.

La presencia de ocho tarjetas en la web indica ocho canales previstos por el modelo, no ocho sensores instalados. En la versión revisada, USE_DHT_SENSOR, USE_HCSR04_SENSOR y USE_WATER_SENSOR son true; USE_PH_SENSOR es false. Esto prueba habilitación por software, no cableado ni calibración correctos.

Qué pasa al recibir un lote parcial

applyLivePayload actualiza únicamente las lecturas válidas que recibió. Los demás indicadores conservan su snapshot anterior. Como seedState crea datos de demostración para todo el catálogo, pueden coexistir lecturas reales y valores iniciales simulados aunque dataMode global ya diga esp32-direct. El campo source de cada lectura es más preciso que la etiqueta global.

Estados de operación, no control físico

El firmware envía un elemento esp32-link dentro de actuators con el número de secuencia. Es un indicador de enlace, no una bomba. El backend también contiene reglas derivadas para bomba, dosificador, aireación y luces en demostración. El endpoint de cambio de actuador solo funciona en modo mock; no existe una ruta de órdenes desde Node hacia relés de la ESP32-S3.

Interpretación de rangos
Los rangos son valores incorporados al prototipo. Deben adaptarse al cultivo, etapa y sensores usados con criterio técnico; este documento los describe, no los prescribe como condiciones agronómicas universales.

MEMORIA TÉCNICA / 12

Firmware de sensores: funcionamiento

Base de implementación: firmware/esp32_web_direct/esp32_web_direct.ino

El firmware de la ESP32-S3 incluye DHT.h, HTTPClient.h y WiFi.h. secrets.h aporta la red inalámbrica y WEB_HOST, que debe ser la dirección alcanzable de la PC. WEB_PORT vale 4173 y la ruta de publicación es /api/ingest/batch. La placa actúa como cliente HTTP, no como servidor de la aplicación.

Señal	GPIO / configuración	Finalidad
DHT11 DATA	4	Humedad y temperatura ambiente.
HC-SR04 TRIG	5	Pulso de disparo ultrasónico.
HC-SR04 ECHO	6	Duración de retorno para estimar distancia.
Agua analógica	7 / ADC de 12 bits	Respaldo de nivel si no hay distancia válida.
pH opcional	1 / deshabilitado	Entrada analógica prevista para el módulo de pH.

setup y loop

setup inicia Serial a 115200 baud, configura pines, inicializa el DHT11 y ajusta lectura analógica a 12 bits. loop intenta conectar WiFi y toma un conjunto de lecturas cuando transcurrieron 5000 ms desde la toma anterior. El esquema usa millis para espaciar ciclos, pero las funciones de lectura y conexión contienen esperas bloqueantes; no es un planificador de tiempo real.

connectWifi evita intentar reconectar más seguido que cada 10 segundos y espera hasta 8 segundos por intento. USE_STATIC_IP es false, por lo que las direcciones estáticas escritas en el código no se aplican por defecto. El router asigna una dirección a la placa.

Lectura y publicación

Las variables se inicializan en NAN. Solo se agregan al JSON lecturas que no sean NaN. Se mide primero distancia y se usa el sensor analógico de agua como respaldo si no hubo eco. Si no hay ninguna lectura válida, no se publica el lote. Los resultados de HTTP y la respuesta del servidor se imprimen por serial para diagnóstico.

El número de secuencia aumenta en cada ciclo y se informa como texto en esp32-link. No constituye una clave de idempotencia. Si el POST falla no se conserva una cola de mediciones para reenviarlas; el siguiente ciclo toma datos nuevos. El firmware actual no agrega el token de ingesta opcional que admite el backend.

Conexión física
La tabla documenta el mapeo del programa, no reemplaza un esquema eléctrico. Se deben comprobar alimentación, masas y niveles admitidos por la placa y cada módulo antes de conectar sus señales.

MEMORIA TÉCNICA / 13

Conversión de mediciones y calibración

Base de implementación: Firmware de sensores: readDistanceCm, waterLevelFromDistance, voltageToPh

Distancia y porcentaje de agua

La medición ultrasónica genera un pulso de 10 microsegundos y espera ECHO mediante pulseIn con timeout de 30000 microsegundos. Si no hay pulso retorna NaN. La fórmula divide por dos porque el recorrido corresponde a ida y vuelta:

```
distancia_cm = duracion_us * 0.0343 / 2
nivel_pct = (25.0 - distancia_cm) / (25.0 - 5.0) * 100
nivel_pct = constrain(nivel_pct, 0, 100)
```

5 cm representa tanque lleno y 25 cm vacío en esta configuración. Con 15 cm la fórmula produce 50%. Son referencias geométricas del montaje: deben ajustarse a la posición del sensor y al recipiente. No se almacena la distancia cruda en el contrato habitual del dashboard.

Respaldo analógico

readWaterRaw promedia 20 lecturas separadas por 5 ms. El resultado se transforma mediante map de 0 a 2500 cuentas hacia 0 a 100% y se limita al intervalo. Es una aproximación por dos extremos; no demuestra una respuesta lineal física. En Arduino map opera con enteros, por lo que este respaldo tiene pasos discretos de porcentaje.

pH previsto, pero desactivado

El código promedia 40 lecturas separadas por 10 ms, convierte cuentas ADC con referencia nominal de 3.3 V y 4095 como máximo, y compensa un divisor resistivo nominal de 10 kΩ y 20 kΩ. Después aplica una recta de dos puntos.

```
V_gpio = raw * 3.3 / 4095
V_modulo = V_gpio * (10000 + 20000) / 20000
m = (8.30 - 2.80) / (2.534 - 3.243)
b = 2.80 - m * 3.243
pH = m * V_modulo + b
```

Las constantes reflejan una calibración escrita en el firmware, no una calibración verificada durante la elaboración del PDF. Su reutilización con otra sonda o módulo no está garantizada. Tampoco hay compensación térmica, registro de calibración ni rechazo estadístico de muestras atípicas.

Por qué conservar datos crudos sería útil

Como mejora, guardar distancia, cuentas ADC, voltaje, estado de validez y fecha de calibración permitiría distinguir un cambio del cultivo de una falla de cableado o deriva del sensor. Hoy la web recibe principalmente el valor convertido y su identificador.

MEMORIA TÉCNICA / 14

Firmware de cámara y configuración

Base de implementación: esp32_cam_webserver.ino; board_config.h; camera_pins.h; app_httpd.cpp

La cámara usa el perfil de placa AI Thinker ESP32-CAM y código basado en CameraWebServer del core Arduino ESP32. La configuración del proyecto y su documentación apuntan al sensor OV3660. El comentario genérico de board_config.h menciona OV2640; el nombre de la placa fija el mapeo de pines, no certifica por sí mismo qué sensor está instalado.

Inicialización de captura

camera_config_t recibe los pines de datos, reloj, sincronismo, control SCCB y alimentación. XCLK se fija en 20 MHz, el formato en JPEG y el perfil inicial en UXGA 1600 × 1200 con quality 10. Si existe PSRAM se usan dos buffers y CAMERA_GRAB_LATEST; el caso inicial usa CAMERA_GRAB_WHEN_EMPTY.

El bloque sin PSRAM intenta usar VGA, DRAM y calidad 12. Sin embargo, después de inicializar se vuelve a solicitar STREAM_FRAME_SIZE y STREAM_JPEG_QUALITY al sensor. Por lo tanto, no conviene presentar ese fallback como una garantía de funcionamiento sin PSRAM en todos los equipos.

Grupo	Pines del perfil AI Thinker
Datos D0 a D7	5, 18, 19, 21, 36, 39, 34, 35
Relojes / sincronismo	XCLK 0; PCLK 22; VSYNC 25; HREF 23
Control de sensor	SIOD 26; SIOC 27; PWDN 32; RESET -1
LED de iluminación	GPIO 4; PWM de 5 kHz y resolución de 8 bits

Si se detecta PID OV3660 se aplican vflip, brillo y saturación específicos. Más tarde, el backend puede cambiar brillo, contraste, saturación y exposición relativa al entrar en un perfil de foto o video. Un parámetro visto al arrancar no necesariamente será el de una captura posterior.

Red y servidores internos

El firmware inicia WiFi, desactiva el ahorro de energía WiFi y espera conectado antes de abrir los servidores. El puerto HTTP base atiende control y fotos; el segundo servidor incrementa el puerto en uno para el stream. En la configuración usual son 80 y 81. loop solo espera; los handlers del servidor atienden solicitudes.

Foco óptico
Este firmware no implementa un mecanismo de autofocus óptico. Cambiar calidad JPEG, resolución o una métrica digital no mueve la lente. La capacidad de ajuste mecánico depende del módulo físico.

MEMORIA TÉCNICA / 15

Protocolos y comunicaciones

Base de implementación: server.js; firmware de sensores; app_httpd.cpp

Enlace	Protocolo y dirección	Contenido
ESP32-S3 a Node	WiFi + HTTP POST	JSON con lecturas y estado esp32-link.
Navegador a Node	HTTP GET / POST	HTML, CSS, JS, bootstrap, órdenes de interfaz y métricas.
Node a navegador	SSE sobre conexión HTTP	Evento snapshot con estado completo; keep-alive.
Node a cámara	HTTP GET	/status, /control, /capture y /stream.
Cámara a Node	JPEG o multipart MJPEG	Binarios comprimidos de imagen.
PC a placas por USB	Programación / serial	Carga del firmware y mensajes de diagnóstico.

SSE no es WebSocket

Server-Sent Events mantiene una conexión de servidor a navegador. La aplicación envía cambios de interfaz en peticiones POST separadas. El servidor almacena respuestas abiertas en CLIENTS y escribe eventos de texto; al desconectarse un cliente lo elimina del conjunto.

```
event: snapshot
data: {"sensors":[...],"snapshot":{...},"meta":{...}}

: keep-alive
```

Las líneas anteriores muestran el formato de transporte; los puntos suspensivos son una abreviatura documental. En el sistema se envía JSON completo. No se implementa un broker MQTT aunque el catálogo use nombres jerárquicos como huerta/sensores/agua/ph. Esos nombres funcionan como identificadores lógicos y alias de entrada.

Por qué hay un proxy de cámara

El navegador pide /api/camera/capture o /api/camera/stream al mismo servidor de la web. Node abre la URL privada del dispositivo y copia la respuesta. Esto concentra la configuración y evita que el cliente tenga que gestionar directamente el origen de los píxeles para Canvas. El dispositivo sigue necesitando ser alcanzable desde la máquina donde corre Node.

localhost siempre designa el equipo desde el cual se interpreta. En secrets.h de la ESP32-S3 debe usarse el host de la PC, no localhost. Una página pública no adquiere acceso a una cámara privada solo por conocer su dirección de red.

MEMORIA TÉCNICA / 16

API de lectura, estado e ingesta

Base de implementación: server.js: enrutador HTTP, applyLivePayload, isAuthorizedIngest

Método y ruta	Respuesta / comportamiento
GET /api/health	200 con ok, port, dataMode, updatedAt, lastIngestAt y cameraMode. No mide la salud física de todos los sensores.
GET /api/bootstrap	200 con estado público completo, incluida configuración de cámara y análisis más reciente.
GET /api/integration/topics	Catálogo de sensorId, label, unit y topic. No abre una conexión MQTT.
GET /api/stream	200 text/event-stream; primer snapshot inmediato y luego actualizaciones.
POST /api/ingest/reading	Acepta una lectura o la forma de lote manejada por el mismo procesador.
POST /api/ingest/batch	Acepta readings y opcionalmente actuators. 200 si hubo entrada aceptable; 400 si no; 401 con token incorrecto.

Ejemplo de mensaje enviado por la placa

```
{
  "source": "esp32-direct",
  "readings": [
    {"sensorId": "air-temperature", "value": 24.5},
    {"sensorId": "air-humidity", "value": 67.0},
    {"sensorId": "water-level", "value": 74}
  ],
  "actuators": [
    {"id": "esp32-link", "label": "ESP32 directa",
     "active": true, "detail": "Ultima secuencia 42"}
  ]
}
```

resolveSensor acepta sensorId o un channel/topic reconocido. normalizeNumber acepta números finitos y cadenas numéricas; se admiten alias value, payload, reading y data. Las entradas inválidas del lote se omiten sin devolver un listado por índice. Un arreglo actuators presente permite aceptar el mensaje aun sin actualizar sensores.

Las fechas inválidas o ausentes se sustituyen por la hora del servidor. La ingesta cambia dataMode y detiene el mock. INGEST_TOKEN es opcional: si está vacío no hay autenticación de ingesta; si existe se verifica x-ingest-token o Authorization: Bearer. El firmware revisado requiere una modificación antes de poder enviar ese token.

MEMORIA TÉCNICA / 17

API de cámara, visión y controles

Base de implementación: server.js: rutas de cámara, análisis y simulación

Método y ruta	Semántica
GET /api/camera/capture	Aplica perfil de foto y transmite JPEG. 404 si no hay modo MJPEG; 409 si hay análisis pendiente; 502 ante falla upstream.
GET /api/camera/stream	Aplica perfil de video y transmite multipart. Se cierra el upstream cuando cierra el consumidor.
POST /api/plant-vision/capture	Obtiene una ráfaga, guarda JPEG y devuelve {ok, capture}. Puede responder 409 o 502.
GET /api/plant-vision/images/<archivo>	Entrega un archivo local de captura como JPEG. 404 si no existe.
POST /api/plant-vision/analysis	Guarda el cuerpo recibido como análisis, agrega timestamp si falta y difunde estado.
POST /api/control/toggle-simulation	Pausa/reanuda mock; si estaba en modo real vuelve al modo mock.
POST /api/control/inject-critical	Introduce un extremo simulado. 409 si está activa la ingesta real.
POST /api/control/actuator	Recibe {id}; alterna override solo en modo mock. Si no procede devuelve 409.

Respuesta de captura: campos principales

```
{
  "ok": true,
  "capture": {
    "imageUrl": "/api/plant-vision/images/plant-<id>-1.jpg",
    "capturedAt": "<fecha ISO>",
    "bytes": 123456,
    "contentType": "image/jpeg",
    "lighting": {"mode":"flash", "intensity":24,
                 "protocol":"settled-burst-v1"},
    "frames": ["<metadatos de cada una de las 3 fotos>"],
    "burstId": "<identificador del lote>"
  }
}
```

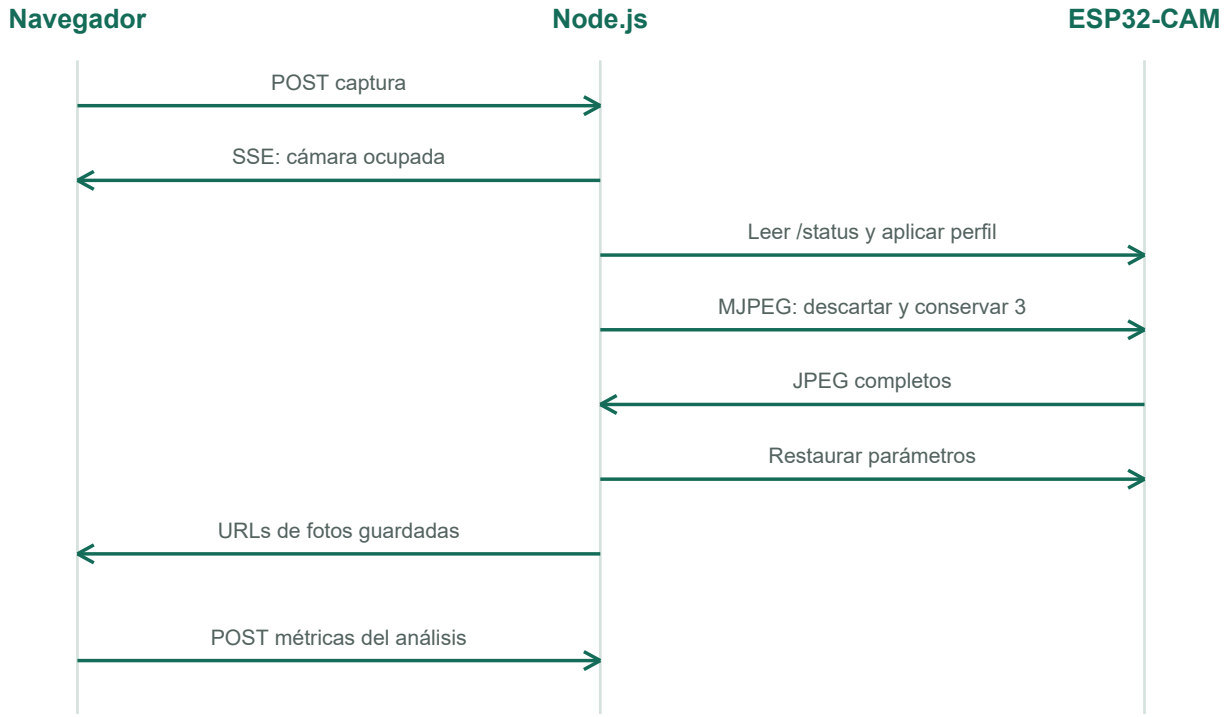
Ejemplo esquemático: frames contiene objetos de captura, no cadenas. bytes y fechas no representan una captura de esta sesión.

El análisis guardado agrega label, status, detail y metrics con versión, ROI, porcentajes, regiones, calidad, estabilidad y tendencia. La API actual confía en métricas calculadas por el cliente: no vuelve a ejecutar el detector para certificarlas ni comprueba estrictamente su correspondencia con una captura.

MEMORIA TÉCNICA / 18

Captura reproducible: secuencia completa

Base de implementación: server.js: capturePlantImage; app/camera-burst.js



Un protocolo independiente de la vista previa

La causa de diferencias entre analizar video iluminado y foto sin flash se aborda fijando un protocolo de adquisición. La petición de análisis no toma simplemente lo que estaba visible en pantalla: abre una ráfaga temporal con parámetros controlados. Así evita depender del modo previo elegido por el usuario.

Node anuncia que hay análisis, cierra consumidores de cámara y espera 500 ms. Lee los valores originales de cada ajuste, aplica resolución de foto, calidad de análisis, color y LED. Con el stream activo descarta al menos seis fotogramas y conserva imágenes solo después de un mínimo de 2000 ms desde el inicio de la solicitud.

Conserva tres JPEG completos y cierra la conexión. Espera otros 500 ms para que el dispositivo libere recursos y restaura los parámetros modificados. Solo después escribe las fotos y devuelve sus URLs. Un bloque finally limpia el estado de ocupado, invalida el perfil cacheado y notifica por SSE.

Qué se estabiliza y qué no
Se estandarizan ajustes solicitados y un tiempo mínimo de espera. El algoritmo no bloquea físicamente la planta ni garantiza una exposición idéntica; las tres fotos todavía deben superar comprobaciones de calidad y comparabilidad.

MEMORIA TÉCNICA / 19

Parser MJPEG, concurrencia y recuperación

Base de implementación: app/camera-burst.js; server.js: queueCameraOperation

No dividir una imagen por un delimitador supuesto

Una respuesta MJPEG contiene múltiples partes: encabezados de texto y un JPEG binario en cada una. Los paquetes de red pueden fragmentar cualquier punto de esa estructura. createFrameParser acumula bytes, localiza el fin del encabezado y obtiene Content-Length; solo entrega un fotograma cuando reunió exactamente esa cantidad de bytes.

```
Cabecera multipart + Content-Length: N + CRLF + CRLF
[ N bytes de JPEG ]
Cabecera del siguiente fotograma...
```

El parser exige longitud válida, limita cada fotograma a 4 MiB y el encabezado a 8192 bytes. Comprueba marcadores iniciales FF D8 y finales FF D9. Estas verificaciones rechazan JPEG truncados, pero no equivalen a validar todos los bloques internos de compresión; la decodificación posterior todavía puede fallar.

Ráfaga acotada

collectCameraBurst usa un timeout total de 30 segundos por defecto. Rechaza respuestas HTTP distintas de 200, desconexiones y streams que terminan antes de reunir el lote. Al completar, cierra respuesta y request para no dejar video ocupando recursos. Los valores genéricos de calentamiento del helper son reemplazados por los más conservadores del servidor.

Exclusión de operaciones

queueCameraOperation encadena promesas. Una operación fallida no rompe la cadena de futuras operaciones porque se captura el rechazo de la cola. cameraAnalysisPending se fija antes de encolar el análisis; una segunda solicitud concurrente recibe 409. Los streams ya abiertos se registran en activeCameraResponses para cerrarlos antes de cambiar el perfil.

La cola serializa tareas de configuración y preparación, no funciona como un gestor multiusuario de sesiones de cámara de larga duración. Su alcance es un proceso Node; múltiples procesos o aplicaciones ajenas que controlen la cámara no quedan coordinados por esta estructura.

Restauración defensiva

withRestoredSettings exige conocer los valores previos antes de modificar nada. Registra incluso el ajuste cuyo request puede haber fallado por timeout, ya que la cámara podría haberlo aplicado. Restaura en orden inverso con dos intentos por parámetro. Si la restauración falla, el error se hace visible: no se informa éxito silencioso.

MEMORIA TÉCNICA / 20

Visión: preparación y segmentación foliar

Base de implementación: app/scripts/plant-vision.js: analyze, integral, morphology, components

El motor es visión por computadora heurística, no un modelo entrenado. Recibe un arreglo RGBA, dimensiones y ROI. En la web las fotos se muestrean a un máximo de 1024 píxeles de ancho conservando proporciones; por ejemplo, QXGA pasa de 2048 × 1536 a 1024 × 768 para el análisis. El JPEG original permanece guardado.

1. Validación y compensación limitada de blanco

Se exige ancho y alto enteros de al menos 16, hasta cuatro millones de píxeles y longitud RGBA exacta. La ROI debe estar dentro de la imagen. En el borde externo del 12% se buscan muestras claras casi neutras; con más de 100 se usan medianas por canal. Las ganancias se limitan a 0.8-1.25 para evitar una corrección excesiva.

```
Rrel = R_corregido / max(G_corregido, 1)
Brel = B_corregido / max(G_corregido, 1)
L = (0.2126*R + 0.7152*G + 0.0722*B) / 255
S = (max(R,G,B) - min(R,G,B)) / max(R,G,B)
```

L es una aproximación de brillo aplicada a los canales disponibles; el código no realiza una conversión colorimétrica completa a luminancia lineal.

2. Contexto verde

Un píxel candidato verde cumple Rrel < 0.97, Brel < 0.98, suma < 1.84, saturación > 0.09 y brillo > 0.055. Se calcula un contexto sobre toda la imagen y una máscara restringida a la ROI. La apertura morfológica elimina estructuras estrechas antes de aceptar superficies amplias.

3. Componentes y reconstrucción

components usa vecinos en ocho direcciones y una cola para agrupar regiones. Se filtran área, compacidad, lado mínimo y elongación. morphology usa imágenes integrales para sumar vecindades: erosión conserva píxeles rodeados de máscara y dilatación expande presencia. Apertura es erosión seguida de dilatación; cierre invierte ese orden.

Un histograma de colores alejados del verde estima el fondo. La reconstrucción puede recuperar zonas no verdes próximas a una hoja, pero evita colores parecidos al fondo para no tratar agujeros como daño. No existe identificación semántica de cada hoja ni seguimiento de su identidad entre capturas.

MEMORIA TÉCNICA / 21

Visión: detección de manchas

Base de implementación: plant-vision.js: referencias locales, candidates y regiones

Comparar con el verde de la propia escena

La máscara foliar se agrupa nuevamente y se calculan medianas de Rrel, Brel y brillo para cada componente. Una referencia local adicional promedia Rrel de píxeles verdes cercanos mediante imágenes integrales. El umbral se adapta parcialmente a esa referencia en lugar de depender solo de un color absoluto de mancha.

Familia de candidato	Criterio conceptual implementado
Cálido / marrón	Rrel supera la referencia verde y un mínimo absoluto; Brel no es alto y hay saturación suficiente.
Cambio hacia amarillo	Rrel crece más respecto del verde y Brel cae. Esto pertenece al detector de manchas y no sustituye el detector de hojas amarillas.
Oscuro poco saturado	Dentro de la hoja: brillo menor que la referencia, desplazamiento de relaciones de color y saturación baja.
Exclusiones	Verde más oscuro por sí solo no cuenta como lesión; zonas demasiado claras con baja saturación se descartan como posible reflejo.

De píxel a región

Un píxel crudo debe tener apoyo en una vecindad de 3 × 3. Las regiones conectadas muy pequeñas o excesivamente alargadas se descartan. La superficie mínima inicial de candidato es max(8, N × 0.00003), donde N es el total de píxeles de la imagen muestreada. Con 1024 × 768, ese valor es aproximadamente 24 píxeles.

Todavía no se dibuja todo candidato en rojo. La fusión posterior exige una región suficientemente ancha, repetición espacial y apoyo dentro de la superficie de hoja. Halos finos de borde y marcas cortadas por la selección pueden permanecer celestes incluso si son visibles en varias imágenes.

Porcentaje con denominador explícito

```
manchas_pct = 100 * pixeles_de_manchas_repetidas
                  / pixeles_de_mascara_foliar
```

Es una proporción dentro de la máscara construida por el algoritmo, no dentro de toda la planta. Se excluye lo oculto, fuera de campo o no segmentado. Una máscara imperfecta cambia el denominador y puede alterar el porcentaje aunque el daño real no haya cambiado.

> **Interpretación correcta**
> La punta marrón reconocida en las pruebas es una señal visual repetida. Su reconocimiento no demuestra presencia de una plaga determinada. Iluminación, pigmentación natural, tejido seco, carencias y otros factores pueden producir colores similares.

MEMORIA TÉCNICA / 22

Visión: amarillamiento de hojas

Base de implementación: plant-vision.js: yellowRaw, yellowGroups, yellowDiagnostics

Una hoja muy amarilla puede quedar fuera de la reconstrucción basada en verde. Por eso el amarillamiento tiene una máscara independiente, evaluada primero en la imagen completa. La ROI se aplica después de reconocer cada componente, para no fragmentar su forma antes de decidir si parece una superficie foliar.

Color, forma y contexto

```
1.02 < Rrel < 1.48
Brel < 0.86
S > 0.18
0.38 < L < 0.94
```

Estos son límites heurísticos después de la compensación de blanco. Una apertura de radio aproximado ancho/300 elimina ruido fino. Los componentes deben tener área mínima max(40, N × 0.0015), compacidad entre 0.18 y 0.9, lado mínimo suficiente y relación de lados no mayor que 8. El límite superior de compacidad intenta reducir objetos rectangulares compactos, pero no reconoce semánticamente una hoja.

La proximidad se calcula contra verde visible después de apertura y antes de una reconstrucción restrictiva. La vecindad se expande aproximadamente un 8% del ancho. El apoyo requerido no crece sin límite con el área amarilla: se limita con el lado corto de la región, porque una hoja grande puede tocar verde solo por un borde.

```
apoyo_minimo = max(20, min(area_region * 0.10,
                           lado_corto_region * 2))
```

Por qué a veces no muestra un porcentaje

Si más del 10% de los píxeles de una región tienen algún canal original en 252 o más, se marca como color saturado no cuantificable. Si la selección corta la región, se conserva el motivo selection-edge. Si no se repite suficientemente, queda not-repeated. yellowOutsidePixels permite avisar que la selección está dejando tejido amarillento fuera.

Denominador diferente del de manchas

```
amarillo_pct = 100 * area_amarilla_repetida
                   / area_union(mascara_foliar, amarillo_repetido)
```

La unión evita duplicar píxeles. El rojo puede tapar visualmente parte del naranja sin borrarla del cálculo. Los porcentajes de manchas y amarillo no deben sumarse: pueden superponerse y usan denominadores diferentes. La máscara utilizada para comparar área temporal no incorpora automáticamente toda región amarilla.

MEMORIA TÉCNICA / 23

Nitidez, calidad y foco: qué mide el software

Base de implementación: plant-vision.js: quality, measureDetail; app.js: drawPlantPreview

Calidad de una fotografía

La calidad se calcula sobre la máscara foliar, no sobre el fondo completo. Se descarta una imagen con poca hoja, oscuridad importante, exceso de exposición o detalle insuficiente según la métrica de gradientes. Esto evita que una fotografía no evaluable se presente como evidencia de ausencia de daño.

Control	Condición actual
Superficie insuficiente	Menos de max(150, N × 0.003) píxeles de hoja.
Subexposición	Brillo promedio menor que 17/100 o más de 25% de píxeles muy oscuros.
Sobreexposición	Brillo promedio mayor que 88/100 o más de 10% de píxeles con canal recortado.
Desenfoque heurístico	RMS de gradientes de paso 2 menor que 3.5 dentro de la hoja.

Detalle hoja: métrica adicional

measureDetail promedia brillo en ventanas 3 × 3 y erosiona cuatro píxeles el interior foliar. Después calcula un Laplaciano de paso 2 en muestras separadas dos píxeles. La erosión reduce el peso del contorno y el promedio limita la influencia del ruido fino. Si hay menos de 25 muestras válidas, el detalle se devuelve como null.

```
lap = L(x-2,y) + L(x+2,y) + L(x,y-2) + L(x,y+2) - 4*L(x,y)
detalle = 255 * sqrt(promedio(lap * lap))
```

El detalle se usa para priorizar fotografías comparables con evidencia suficiente. No se debe confundir con sharpness, la métrica de gradiente usada para el rechazo inicial. Tampoco es un porcentaje, una distancia de enfoque o una probabilidad de acierto. Una hoja con más nervaduras puede tener mayor detalle que otra, aun con igual foco óptico.

Qué mejora y qué no

El programa puede elegir una imagen con más detalle, controlar iluminación y compresión, advertir problemas y ampliar el área de interés. No puede recuperar información que la lente no captó ni enfocar ópticamente un módulo sin mecanismo controlable. No se aplica realce artificial a los píxeles antes de medir manchas.

Ensayo documentado
La prueba sobre una imagen real con desenfoque sintético creciente produjo detalle aproximado 3.72, 2.97, 2.03, 1.21 y 0.50. Demuestra sensibilidad a ese desenfoque, no una calibración universal de foco.

MEMORIA TÉCNICA / 24

Fusión de tres fotos y repetibilidad

Base de implementación: plant-vision.js: compareFrames y combine

No promediar cualquier conjunto de imágenes

combine analiza qué fotografías son comparables antes de acumular evidencia. Cada foto genera una firma de brillo en una grilla 32 × 24. compareFrames exige dimensiones iguales, presencia de hoja, correlación de firmas mayor que 0.985, intersección sobre unión de máscaras mayor que 0.65 y diferencia de brillo promedio menor que 12.

```
IoU = area(interseccion(mascara_A, mascara_B))
      / area(union(mascara_A, mascara_B))
```

No hay registro geométrico, flujo óptico ni corrección de perspectiva. Si la hoja se mueve demasiado, se rechaza la comparación. Los umbrales toleran variación moderada, pero pueden fallar ante deformaciones, cambios de fondo o iluminación compleja.

Selección de fotografía de referencia

Prioridad de ranking	Ponderación del código
Imagen con calidad utilizable	10000 puntos.
Cantidad de pares comparables	1000 por fotografía utilizable y comparable.
Apoyo repetido de manchas	100 si se supera el apoyo mínimo.
Apoyo repetido de amarillo	50 si se supera el apoyo mínimo.
Detalle interior	Hasta 40 puntos.
Puntaje general de calidad	score × 0.01 como aporte menor.

Esta jerarquía favorece consistencia y evidencia antes que un borde artificialmente nítido. La mejor foto no necesariamente es la de mayor detalle absoluto del lote. Con dos fotos comparables puede haber resultado utilizable aunque una tercera se haya movido.

Votación espacial

Las máscaras candidatas se dilatan ligeramente para tolerar desalineación pequeña. La votación requiere max(2, techo(0.6 × fotos comparables)) apoyos. En una ráfaga de tres normalmente bastan dos votos por píxel. Las regiones de manchas exigen además forma y superficie, y al menos 45% de apoyo espacial; las amarillas exigen al menos 65%, color evaluable y no estar cortadas.

Una sola imagen no puede aportar repetibilidad. El resultado usable combina calidad y estabilidad, no solo número de candidatos. La repetición de un reflejo o distractor persistente sigue siendo posible: consistencia temporal reduce artefactos transitorios, pero no sustituye una validación semántica.

MEMORIA TÉCNICA / 25

Comparación de área y crecimiento

Base de implementación: app/scripts/plant-trends.js

plantAreaPercent es el porcentaje de la imagen muestreada ocupado por la máscara foliar. Cuando hay ROI, la máscara se restringe a ella pero el denominador sigue siendo el tamaño de imagen completo. No debe interpretarse como porcentaje de la selección ni como área física en cm².

Selección de una referencia compatible

compareHistory busca la referencia más reciente entre una hora y siete días anteriores. No usa fotos de la misma ráfaga ni referencias que compartan URL o fecha de captura. Exige versión del algoritmo, dimensiones y ROI equivalentes, iluminación con igual protocolo, intensidad y modo, y mediciones válidas con al menos dos fotos comparables.

También exige que la hoja no toque el borde de imagen/ROI, desplazamiento del centro menor que 0.015 en coordenadas normalizadas, cambios de ancho y alto menores que 0.04 y diferencia de brillo menor que 10. Son tolerancias del prototipo, no parámetros calibrados contra crecimiento físico conocido.

Cambio por encima de la variación observada

```
rango_cambio = [minActual - maxBase, maxActual - minBase]
detectable = rango_cambio.min > 0.5
             OR rango_cambio.max < -0.5
delta_pp = areaActual - areaBase
```

Cada ráfaga aporta un mínimo y un máximo de área. Para declarar aumento o disminución, todo el intervalo de diferencia debe quedar fuera de ±0.5 puntos porcentuales. Si no ocurre, la tendencia es No concluyente y el cambio numérico queda null. Eso no afirma que no hubo crecimiento; dice que este método no distingue el cambio de su variación observada.

Ejemplo ilustrativo	Resultado
Base 20.0-20.4%; actual 20.2-20.7%	Intervalo -0.2 a +0.7 pp: no concluyente.
Base 20.0-20.4%; actual 21.1-21.5%	Intervalo +0.7 a +1.5 pp: aumento detectable de área proyectada.
Misma escena, cambio de ROI	No hay referencia compatible aunque el número de área parezca mayor.

No es un intervalo estadístico
El rango usa variación entre unas pocas fotos del mismo lote, no un intervalo de confianza. Postura, solapamiento, pérdida de verde y perspectiva alteran el área visible. Para medir crecimiento físico hacen falta escala, geometría y validación temporal independientes.

MEMORIA TÉCNICA / 26

Persistencia, imágenes y trazabilidad

Base de implementación: server.js: loadState, persistState, normalizeState, savePlantVisionAnalysis

Qué queda almacenado

data/runtime-state.json contiene snapshot, historial de sensores, alarmas, metadatos y análisis. data/plant-captures guarda los JPEG; los análisis conservan sus URLs y datos de iluminación, no copias binarias dentro del JSON. Esto permite volver a mostrar una foto después de recargar la página.

loadState crea datos simulados cuando no existe el archivo. Si existe, normalizeState limita historiales y completa campos faltantes. Si falla lectura o parseo, el código genera un estado nuevo y lo escribe. Hoy no mantiene automáticamente una copia del archivo corrupto para recuperación posterior.

Persistir no equivale a una transacción de base de datos

persistState serializa todo el objeto y lo sobrescribe de manera síncrona. No hay una operación temporal más rename atómico, journal ni control de concurrencia entre procesos. Una interrupción durante la escritura puede dejar el archivo incompleto. Una instancia adicional del servidor compartiendo la carpeta también puede sobrescribir datos de otra.

Límites de retención

Se conservan 40 muestras por sensor y 40 entradas de análisis, además de hasta 20 alarmas. El primer límite es por cantidad, no por duración: a un dato cada cinco segundos cubre aproximadamente tres minutos y veinte segundos. Las cuarenta muestras del mock corresponden aproximadamente a ochenta segundos.

Las imágenes no se borran automáticamente cuando una entrada sale del historial. El disco puede acumular archivos huérfanos o fotografías que ya no aparecen en las últimas cuarenta entradas. El conjunto histórico disponible también limita si habrá una referencia válida para comparación de área.

Versionado del resultado

analysisVersion evita mezclar métricas de algoritmos distintos en la tendencia. Las fotos anteriores pueden reanalizarse para generar una vista con el motor actual, pero no debe confundirse esa superposición con una migración certificada de todas las métricas guardadas. El cliente identifica resultados de otra versión como análisis anterior.

> **Respaldo y privacidad**
> Las fotos y el estado son datos locales excluidos de Git. Un clone del repositorio no recupera el historial del cultivo. Para respaldo consistente conviene detener el proceso y copiar estado más imágenes a un destino privado; las credenciales deben mantenerse aparte.

MEMORIA TÉCNICA / 27

Patrones y decisiones de diseño

Base de implementación: Interpretación del código, no atribución histórica de intención

Patrón o principio	Manifestación concreta y límite
Cliente-servidor por capas	Presentación en navegador, integración y persistencia en Node, adquisición en firmware. No son microservicios independientes desplegados.
Módulo / encapsulación	IIFE en app.js y charts.js; envoltorio compatible con navegador y CommonJS en visión y tendencias.
Publicación-suscripción	SSE: CLIENTS recibe snapshots emitidos por broadcast. No hay broker externo ni almacenamiento de eventos.
Proxy	Node intermedia recursos de la cámara y oculta al consumidor el manejo directo del stream privado.
Adaptación de entradas	resolveSensor y normalizadores aceptan distintos nombres y tipos y los convierten al modelo de lectura.
Cola de operaciones	Cadena de promesas para organizar accesos a una cámara con recursos limitados.
Adquirir-usar-restaurar	withRestoredSettings guarda configuración, captura y restaura aun después de error. Es similar a manejo disciplinado de recursos, no una transacción ACID.
Núcleo de cálculo separado	analyze, combine y compareHistory reciben datos sin depender del DOM o la red; facilitan pruebas deterministas.
Degradación controlada	Mock local ante fallo de bootstrap y estados no evaluables cuando la imagen no alcanza requisitos.

Qué patrones no conviene atribuirle

No hay clases Model, View y Controller formalmente separadas, aunque se distinguen datos y renderizado. No corresponde presentarlo como MVC estricto, arquitectura hexagonal completa o implementación de todos los principios SOLID. El backend sigue concentrado en un archivo y app.js reúne múltiples responsabilidades.

Ventajas y compromisos

Usar APIs nativas reduce dependencias de producción y hace visible todo el recorrido de una lectura. A cambio, validación, autenticación, enrutamiento, accesibilidad y ciclo de vida deben mantenerse manualmente. La duplicación de catálogo y reglas entre backend y fallback del navegador puede derivar en inconsistencias si solo se modifica uno.

MEMORIA TÉCNICA / 28

Configuración del servidor

Base de implementación: Variables leídas por server.js; valores predeterminados

Variable	Valor / función
PORT	4173. Puerto del backend.
INGEST_TOKEN	Vacío. Token opcional solo para endpoints de ingesta.
CAMERA_MODE	Selección opcional: mjpeg, embed o fallback mock.
CAMERA_MJPEG_URL	URL del stream del dispositivo; habilita MJPEG si es válida.
CAMERA_CAPTURE_URL	Opcional. Si falta se deriva /capture del host de stream.
CAMERA_CONTROL_URL	Opcional. Se deriva /control del host de cámara.
CAMERA_STATUS_URL	Opcional. Se deriva /status del host de cámara.
CAMERA_EMBED_URL	URL para iframe. No implica disponibilidad del análisis de ráfaga.
CAMERA_LABEL	ESP32-CAM OV3660.
CAMERA_CAPTURE_REFRESH_MS	3600000: una hora entre refrescos de foto.
CAMERA_ANALYSIS_FLASH_INTENSITY	24; limitado a 0-255.
CAMERA_ANALYSIS_JPEG_QUALITY	8; limitado a 8-63. Menor número implica menor compresión.
CAMERA_ANALYSIS_FLASH_SETTLE_MS	250 en configuración; la captura exige al menos 2000 ms efectivos.

Si el stream usa puerto 81, las URLs derivadas de control y captura quitan ese puerto y utilizan el HTTP predeterminado. Con una cámara servida en otros puertos conviene indicar las URLs explícitamente. La configuración se lee al iniciar el proceso; cambiar variables en otra terminal no reconfigura un servidor que ya está ejecutándose.

Foto y video no tienen el mismo perfil

Ajuste	CAMERA_STILL_*	CAMERA_VIDEO_*
FRAMESIZE	19: QXGA	15: UXGA
QUALITY	8	10
BRIGHTNESS / CONTRAST	0 / 1	0 / 1
SATURATION / AE_LEVEL	0 / -1	0 / -1
LABEL	QXGA 2048x1536 HQ	UXGA 1600x1200

El análisis utiliza framesize y settings del perfil still, pero su calidad JPEG sale de CAMERA_ANALYSIS_JPEG_QUALITY. Los enteros de tamaño son propios del driver; no representan dimensiones en píxeles directamente.

MEMORIA TÉCNICA / 29

Instalación y puesta en marcha

Base de implementación: README.md; firmware/*/README.md; package.json

1. Preparar la aplicación

En una PC nueva se obtiene el repositorio, se instala un entorno Node compatible y se abre la carpeta raíz. El paquete actual no necesita dependencias npm para arrancar. La ruta concreta del repositorio puede cambiar entre máquinas; los comandos siguientes presuponen estar ubicado en ella.

```
node --version
node server.js
```

La aplicación se abre en http://localhost:4173. Para acceder desde otro equipo de la misma red se usa la IP de la PC y el puerto, con las reglas de firewall apropiadas. server.listen no fija exclusivamente 127.0.0.1: no se debe asumir que el servicio es inaccesible desde la LAN.

2. Preparar las placas

Cada carpeta de firmware contiene secrets.h.example para crear su archivo local secrets.h. En sensores se completan red, contraseña y WEB_HOST; en cámara, credenciales de red. No se deben incluir esos valores en commits ni en informes. Los perfiles sketch.yaml documentan placa, core y bibliotecas usados; los puertos USB deben identificarse en cada PC, sin asumir COM5 o COM6.

Con Arduino IDE se seleccionan la placa y puerto correspondientes y se carga cada sketch por separado. Los mensajes seriales a 115200 permiten obtener la IP de cámara, detectar fallos de inicialización y comprobar respuestas HTTP de la ESP32-S3. No basta con que el LED de una placa esté encendido para confirmar ingesta.

3. Iniciar con cámara real

```
# PowerShell: reemplazar el marcador por la IP real de la camara
$env:CAMERA_MJPEG_URL = "http://<IP_CAMARA>:81/stream"
$env:CAMERA_LABEL = "ESP32-CAM OV3660"
node server.js
```

Si ya hay un servidor, no se inicia otro sobre el mismo puerto. Se debe identificar el proceso existente y coordinar su reinicio. Si se cambia PORT también debe alinearse WEB_PORT del firmware de sensores. El navegador no envía automáticamente esa nueva dirección a las placas.

4. Comprobación funcional

Revisar /api/health y /api/bootstrap, verificar source y timestamp por sensor, alternar foto/video y ejecutar una captura. Confirmar que la iluminación vuelve a su estado previo y que el resultado se conserva al recargar. No cambiar una configuración eléctrica ni forzar actuadores físicos como parte de una prueba de interfaz.

MEMORIA TÉCNICA / 30

Pruebas y evidencia de validación

Base de implementación: tests/*.test.js; app/docs/plant-vision-validation.md

Ejecución realizada para este informe: 15 de septiembre de 2026, sobre los archivos locales revisados. Resultado final: **58 pruebas aprobadas, 0 fallidas y 0 omitidas**, con duración aproximada de 29.4 segundos. Se habilitó sharp desde el runtime local y se utilizaron las fotografías ya existentes; no se adquirieron nuevas imágenes del hardware.

Conjunto	Casos	Qué comprueba
camera-burst.test.js	8	Multipart fragmentado, tamaños, JPEG incompleto, calentamiento, desconexión y restauración tras fallos.
plant-trends.test.js	10	Referencias, fechas, compatibilidad, intervalos, recortes, calidad y entradas inválidas.
plant-vision.test.js	27	Verde, manchas, sombras, agujeros, ROI, amarillo, repetición, movimiento y detalle.
real-captures.test.js	13	Regresión sobre fotos del cultivo, punta marrón, amarillo, luz, color y desenfoque.

Cómo repetir

```
node --test tests/*.test.js
# Equivalente definido en package.json:
npm test
```

Sin sharp accesible, la suite omite de forma explícita las pruebas de fotos reales; no las considera aprobadas. La primera ejecución de esta revisión dio 45 aprobadas y 13 omitidas. Al hacer disponible la dependencia y el corpus se ejecutaron las 58. Ese detalle importa al comparar resultados en otra PC o en integración continua.

Herramientas auxiliares

inspect-captures.js genera máscaras y resultados de inspección. audit-vision-history.js vuelve a evaluar entradas guardadas sin reescribir el historial. capture-lighting.js ensaya intensidades de iluminación en hardware y requiere una URL de prueba; modifica temporalmente la cámara y no debe ejecutarse simultáneamente con otros consumidores.

Qué no queda demostrado

La suite no certifica precisión universal en especies distintas, exactitud agronómica ni crecimiento real en centímetros. Las fotos reales proceden principalmente de una planta y escena. No hay porcentaje de cobertura de código medido en este informe ni una batería integral automatizada de rutas HTTP del servidor completo, accesibilidad, seguridad o carga multiusuario.

MEMORIA TÉCNICA / 31

Seguridad y límites antes de publicar

Base de implementación: Revisión estática de server.js y flujos del cliente

Estado actual: prototipo de red de confianza
No conviene publicar el proceso tal como está ni redirigir un puerto del router hacia él. Estas observaciones describen riesgos visibles en el código; no se realizaron pruebas de intrusión ni lectura de archivos secretos por HTTP.

1. Publicación de archivos demasiado amplia

serveStatic usa como raíz toda la carpeta del proyecto y bloquea solo algunos subdirectorios, como data y app/docs. No tiene una lista blanca limitada a index.html, scripts y estilos. Por diseño puede dejar alcanzables archivos de firmware, configuración, metadatos Git o secrets.h si están dentro de esa raíz. Que un archivo esté en .gitignore no impide que un servidor lo entregue por HTTP.

Recomendación: usar un directorio público separado, permitir únicamente los recursos necesarios y excluir dotfiles, código de backend, firmware y secretos. Validar pertenencia mediante rutas resueltas y límites de directorio, no solo una comparación textual de prefijo.

2. Autenticación y permisos incompletos

INGEST_TOKEN protege únicamente ingesta cuando se configura. La captura, la publicación de análisis, los controles y la lectura de datos no tienen autenticación general. El estado público incluye configuración upstream de cámara. Deben agregarse sesiones o credenciales apropiadas, autorización por operación y protección de peticiones que cambian estado.

3. Entradas y datos no confiables

parseBody acumula el cuerpo completo sin límite y trata JSON inválido como objeto vacío. savePlantVisionAnalysis persiste el objeto del cliente con validación mínima. No hay rate limiting ni cuotas de almacenamiento. Algunas vistas construyen HTML dinámico; campos procedentes de dispositivos o clientes requieren revisión de escape para no introducir contenido activo.

4. Transporte, integridad y disponibilidad

El firmware envía HTTP sin TLS. El backend no tiene TLS integrado aunque puede usar https para ciertos upstream. Se necesita protección del transporte en un despliegue real, respaldo, registro de eventos y gestión de secretos. Las solicitudes de captura consumen cámara, CPU y disco; sin límites pueden degradar el servicio.

Para cualquier exposición externa, corregir estos puntos tiene prioridad sobre sumar funciones visuales. El documento registra las recomendaciones; ninguna de ellas fue implementada durante su elaboración.

MEMORIA TÉCNICA / 32

Operación, fallos y diagnóstico

Base de implementación: Comportamientos observados en código; guía de mantenimiento

Síntoma	Qué revisar
La web abre, pero muestra mock	Backend no disponible al iniciar, o no llegó una ingesta válida. Revisar etiqueta y source por sensor.
ESP32 publica, pero hay indicadores congelados	El firmware no implementa todos los canales o omitió una lectura NaN. Revisar timestamp individual, no solo dataMode.
HTTP 401 en la placa	INGEST_TOKEN está activo, pero el firmware no envía la cabecera esperada.
Cámara devuelve 502	IP, alimentación, red, perfil de captura, respuesta upstream y otros consumidores de video.
Captura devuelve 409	Hay análisis pendiente. Esperar y revisar el cliente que lo inició.
Amarillo indica área incompleta	La ROI corta o deja afuera tejido candidato. Revisar selección con imagen completa.
Foto no evaluable	Falta hoja, luz, detalle o hay saturación. No interpretarlo como planta sana.
Cambio de área no disponible	Referencia demasiado reciente, incompatible, recortada, misma captura o variación excesiva.
No se restaura la cámara	Problema de control/red. Revisar estado real del flash y parámetros, no asumir que volvieron solos.
Error al usar el puerto 4173	Ya hay un proceso escuchando o el puerto no está disponible. Identificarlo antes de reiniciar.

Observabilidad existente

La ESP32-S3 imprime lecturas, URL, cuerpo y respuesta HTTP por serial. La cámara registra inicialización y algunos errores. Node informa la URL de inicio. Hay health y panel de servicios, pero no un sistema centralizado de logs estructurados, trazas por captura, alertas externas o métricas de disponibilidad.

No destruir evidencia durante el diagnóstico

El script npm run reset-data elimina runtime-state.json. No es una reparación inocua: borra estado e historial guardado en ese archivo, aunque deja las imágenes. Antes de usarlo se debe respaldar y comprender la causa del fallo. Para reproducir problemas del detector, conservar fotos originales, ROI, versión y parámetros de luz.

MEMORIA TÉCNICA / 33

Rendimiento, escalabilidad y despliegue

Base de implementación: Implicaciones de la implementación actual

Costo de procesamiento

El ancho de muestreo de 1024 reduce el costo frente a analizar todos los píxeles QXGA. El motor usa arreglos tipados, componentes y tablas integrales; varias operaciones recorren la imagen completa. La etapa visual se ejecuta sin Web Worker en el hilo principal del navegador, por lo que puede producir pausas en equipos lentos.

Las imágenes decodificadas consumen más memoria que sus JPEG. Un RGBA de 2048 × 1536 tiene unos 12 MiB sin comprimir; tres pueden representar alrededor de 36 MiB antes de sumar canvas, máscaras y buffers. No es una medición de consumo total del navegador, sino un cálculo de tamaño de píxeles.

Cuellos de botella previsibles

El backend copia streams de cámara, envía snapshots completos por SSE y sobrescribe el JSON completo. El trabajo sincrónico de disco bloquea temporalmente otras solicitudes. El historial es corto, no una base de series temporales. La cola y el estado en memoria pertenecen a un único proceso y no coordinan réplicas.

Separación de despliegue posible

La interfaz estática puede separarse técnicamente del proceso Node, pero sus fetch usan rutas relativas y su contrato presume mismo origen. Un traslado requiere configurar la URL de API, autenticación, CORS cuando corresponda y acceso autorizado a las imágenes. El backend además necesita alcanzar los dispositivos en la LAN.

El diseño actual espera un proceso persistente, conexiones SSE prolongadas y disco con datos durables. Un entorno de funciones efímeras no es un reemplazo directo del servidor sin rediseñar persistencia, conexiones y adquisición. No se ha desplegado esta arquitectura en Vercel como parte de este trabajo, ni se afirman condiciones comerciales o límites actuales de un proveedor.

Mejoras razonables por etapas

Primero, limitar recursos públicos, validar entradas y separar origen real/simulado por sensor. Luego, mover el cálculo visual a un Worker y medir tiempos reales. Para crecer, reemplazar JSON por una base durable, imágenes por almacenamiento controlado y la cola local por coordinación compartida. La elección de infraestructura debe seguir los requisitos medidos, no anticiparlos.

MEMORIA TÉCNICA / 34

Evolución recomendada del proyecto

Base de implementación: Propuestas; no forman parte de la implementación documentada

Prioridad	Trabajo propuesto	Criterio de aceptación
1. Datos confiables	Estado válido, obsoleto o simulado por sensor; reloj de disponibilidad independiente.	Una lectura ausente no se presenta como real reciente.
2. Seguridad	Raíz pública mínima, autenticación, schemas, límites y protección del transporte.	Ninguna ruta pública entrega secretos; solicitudes inválidas se rechazan de forma consistente.
3. Adquisición estable	Montaje rígido, luz difusa repetible, referencia de color y registro de parámetros.	Variación entre capturas medida con escena inmóvil y sesiones distintas.
4. Dataset etiquetado	Fotos de diferentes plantas, fondos, cámaras y condiciones; máscaras revisadas.	Separación por planta/sesión entre ajuste y evaluación, evitando fuga de datos.
5. Segmentación y evaluación	Medir precisión, exhaustividad e IoU por tipo de señal; comparar con el detector actual.	Mejora en conjunto independiente sin ocultar fallos ni omisiones.
6. Crecimiento físico	Escala visible, cámara fija, control de perspectiva y seguimiento de hojas.	Contraste contra mediciones manuales reales durante días/semanas.
7. Operación durable	Base de datos, retención de imágenes, backup y logs estructurados.	Recuperación probada tras reinicio y fallos de almacenamiento.

Si se incorpora aprendizaje automático

Un modelo entrenado tendría que resolver un objetivo concreto: separar hoja/fondo, clasificar síntomas o localizar insectos, que no son la misma tarea. Harían falta etiquetas, entrenamiento o selección de un modelo adecuado, validación independiente y medición de costo. Su salida no debería sustituir sin control las advertencias de calidad y las reglas de adquisición actuales.

Si se incorpora control físico

Agregar relés y dosificación exige diseñar un canal de órdenes separado, permisos, confirmación de ejecución y estados seguros ante pérdida de red. Las reglas visuales no deben activar automáticamente químicos o actuadores críticos. El proyecto actual es de monitoreo y asistencia visual, no de control agronómico autónomo certificado.

MEMORIA TÉCNICA / 35

Glosario técnico

Base de implementación: Términos utilizados en este informe

Término	Definición aplicada al proyecto
Frontend / backend	Código de presentación en el navegador / proceso que sirve datos, archivos y funciones de integración.
API / endpoint	Contrato de comunicación / ruta concreta con método y respuesta definidos.
HTTP / SSE	Peticiones y respuestas web / actualizaciones del servidor sobre una conexión abierta.
JPEG / MJPEG	Formato de fotografía comprimida / secuencia de JPEG dentro de una respuesta multipart.
Firmware	Programa compilado que corre dentro de cada microcontrolador.
GPIO / ADC	Pin digital de propósito general / conversor de señal analógica a número.
PSRAM / framebuffer	Memoria externa utilizada por la cámara / buffer que guarda un fotograma.
ROI	Región de interés definida con coordenadas normalizadas sobre la foto completa.
Máscara / componente	Mapa de píxeles seleccionados / conjunto conectado de esos píxeles.
Morfología	Operaciones espaciales sobre máscaras, como erosión, dilatación, apertura y cierre.
IoU	Área de intersección dividida por área de unión de dos máscaras.
RMS / Laplaciano	Raíz del promedio de cuadrados / operador de variación espacial usado aquí para detalle.
Mock / fallback	Datos simulados / alternativa de funcionamiento al fallar la ruta principal.
Snapshot / burst	Estado actual completo / ráfaga de fotografías obtenidas juntas.
Puntos porcentuales	Diferencia entre dos porcentajes. De 20% a 22% son +2 pp, no +2% relativo.
Regresión / corpus	Prueba que detecta pérdida de comportamiento / conjunto de imágenes usadas en evaluación.

En este proyecto, calidad y detalle son indicadores heurísticos. Estable significa comparable dentro de una ráfaga. Repetida significa que una marca cumple reglas de consistencia. Ninguno de esos términos debe traducirse automáticamente a certeza diagnóstica.

MEMORIA TÉCNICA / 36

Referencias y mapa de lectura del código

Base de implementación: Fuentes primarias: archivos locales revisados al 15/09/2026

La fuente principal de este documento es el código del espacio de trabajo, incluyendo cambios aún no incluidos en el último commit. Las rutas siguientes son relativas a la raíz del repositorio. Los nombres de función permiten ubicar cada implementación incluso si cambia la numeración de líneas.

Fuente	Puntos de entrada recomendados
package.json; README.md	Comandos, panorama general y advertencias del prototipo.
index.html; app/styles/main.css	Estructura visible, controles, tokens y reglas responsive.
app/scripts/app.js	bootstrap, hydrate, connectStream, runPlantVisionAnalysis, makePlantAnalysis, drawPlantPreview.
app/scripts/charts.js	drawLineChart y drawCamera.
server.js	loadState, applyLivePayload, getPublicState, capturePlantImage, serveStatic y http.createServer.
app/camera-burst.js	createFrameParser, collectCameraBurst y withRestoredSettings.
app/scripts/plant-vision.js	analyze, compareFrames, combine y focusCrop.
app/scripts/plant-trends.js	validMeasurement, comparable y compareHistory.
firmware/esp32_web_direct/	Sketch, perfil de compilación y README de adquisición.
firmware/esp32_cam_webserver/	Sketch, app_httpd.cpp, board_config.h, camera_pins.h y perfil.
tests/; app/docs/	Casos automatizados, auditorías e historial documentado de validación.

Notas de consistencia documental

El README conserva una frase anterior que llama amarillas a las candidatas inciertas. La interfaz actual usa celeste para esas candidatas y naranja oscuro para amarillamiento repetido; este informe sigue el código. La cabecera de la guía de validación menciona v4.1, pero sus secciones posteriores documentan v4.4. También se distinguió el comentario genérico OV2640 del perfil de cámara y su configuración orientada a OV3660.

Identidad del repositorio

Nombre de trabajo: **WebHidroponia / HydroVision**. Commit de referencia: **0397600**, “Improve leaf yellowing detection and area comparison”. La edición documentada incluye mejoras locales v4.4 de foco visual, amarillamiento, selección y pruebas. Este PDF no certifica que esos cambios estén publicados en el remoto.

Síntesis
La solución une adquisición embebida, una API Node nativa y un cliente web sin framework. Su principal fortaleza es el flujo trazable desde una lectura o una foto hasta su visualización. Su evolución debe priorizar datos verificables, seguridad y validación independiente antes de prometer diagnóstico o crecimiento físico.